

AI Operations, Module 3 Assignment

Raghava Roll No: DA24B021 September 19, 2026

Q1: Docker image size, naive vs multi-stage

Both images build and run, and both answer `/healthz` and `/predict` the same way, labelling the sample spam message as spam and the ham one as ham.

Measured by	Naive	Multi-stage	Reduction
<code>docker image inspect</code>	584 MB	158 MB	72.9 %
<code>docker images</code>	2.35 GB	770 MB	67.2 %

`inspect` adds up the layers the image itself contributes; `docker images` also counts the base layers underneath. Both give a saving of around 70 percent.

The naive image sits on the full `python:3.12`, so everything pip used during the install is still there at runtime. `which gcc` returns `/usr/bin/gcc`, the dev headers and apt lists from that base are present, and pip's wheel cache is in `/root/.cache` at 91 MB after installing scikit-learn, pandas, numpy and scipy. The `pip install` layer alone is 535 MB. The multi-stage build discards the builder and copies only `/opt/venv` into `python:3.12-slim`, where the same two checks find no `gcc` and no `/root/.cache`. The API loads a joblib file and answers HTTP requests, so none of that was needed once pip had finished.

Q2: Docker Compose with a Redis cache

`/predict` hashes the input text and looks it up in Redis first. A hit returns the stored label without touching the model; a miss runs the model, writes the label with a 300 second TTL, and returns it. Both paths give the same label for the same text, and the response carries a `cached` flag so the two can be told apart.

`docker-compose.yml` defines `api`, built from the multi-stage Dockerfile, and `cache`, running `redis:7-alpine`. The API reaches Redis at the hostname `cache`, which is just the service name, since Compose puts both containers on one network and registers each name in its DNS. No IP addresses appear anywhere in the config.

Ten rounds, a different message each round so the first call is a miss and the second a hit. Every call returned 200.

median cache MISS	1.788 ms
median cache HIT	1.208 ms
speedup	1.48x

Two things keep the gap small. `curl` times the whole HTTP round trip and the cache only skips the model call, and the model is a TF-IDF vectoriser plus Naive Bayes over one short string, which already takes well under a millisecond. A heavier model would show a much larger difference.

Compose versus Kubernetes. A Dockerfile builds one image, and cannot say that two containers belong together, share a network, or that one waits for the other. Compose does that on a single machine. Kubernetes does the same across a cluster and also handles what comes after startup: placing pods on nodes, replacing ones that die, changing versions without dropping requests, and scaling replica counts.

Q3: Indexed Job over 8 shards

The manifest sets `completions: 8` with `completionMode: Indexed`, `restartPolicy: Never`, CPU requests equal to limits at 500m, and `POD_NAME` and `NODE_NAME` pulled in through the Downward API so each pod reports where it ran.

Choosing parallelism. Two nodes at 2 CPUs each is 4000m allocatable, but not all of it is free. `kubectl describe nodes` shows 850m already requested on the control-plane node and 100m on the worker, for kubelet, kube-proxy, CoreDNS and the CNI pods, leaving roughly 3050m. At 500m each, four pods need 2000m and fit, so `parallelism: 4` and the eight shards run in two waves.

An earlier attempt used 900m per pod. Four of those want 3600m, more than the 3050m free, and the fourth pod stayed Pending with `0/2 nodes are available: 2 Insufficient cpu`. The node spec alone is not enough to size this; what is already requested has to come off first.

pods Running at once	4
shards processed	8 / 8
rows checked	800
total invalid rows	154

`kubectl get pods -o wide` caught four pods at once, and the ages show the waves: four at 14 seconds, then two at 6 and two at 4. Per-shard counts came back 18, 17, 21, 20, 16, 17, 24 and 21, matching what `generate_shards.py` planted.

Why the API and not a shared volume. minikube's default provisioner is hostPath backed, so a PersistentVolume lives on one node's filesystem and stays bound there. Nothing in the manifest controls which node a pod lands on, so a pod could easily end up somewhere that volume is not. Each validator prints its result to stdout instead, and `collect_results.py` reads it back with `read_namespaced_pod_log`. The kubelet on each node captures its own pods' output, so the API server can return it wherever the pod ran.

With 6 CPUs. Raise it to 6. 6000m divided by 500m leaves plenty of room, and even allowing for the same system overhead six pods fit comfortably where four did before. Two waves become one of six and one of two. Going past eight changes nothing, since `completions: 8` caps the work.

Q4: Deployment, self-healing and rolling update

Two replicas behind a NodePort Service selecting on `app: spam-api`, with a readiness probe on `/healthz`. That endpoint returns 503 until the model has loaded, so a pod stays out of the Service until it can answer.

Self-healing. Deleting `spam-api-6f5975cf7d-b8n69` left one pod, and `spam-api-6f5975cf7d-v8h85` came up in its place, Running and Ready about 11 seconds later. The Deployment did not do this itself. It creates a ReplicaSet, and the ReplicaSet controller owns the pods: it compares how many pods carry its label selector against `spec.replicas`, and the deletion took that count from 2 to 1, so it made another. The original was not restarted, which is why the replacement has a different name.

Rolling update. `kubectl set image` to v2 moved the Deployment from ReplicaSet `6f5975cf7d` to `68cf7bfbc`, and `rollout history` lists revisions 1 and 2. Part way through, both new pods were Running while the last old one was still Terminating, so the number serving never fell below two. `maxUnavailable: 0` with `maxSurge: 1` forces that order: a new pod passes its readiness probe before an old one goes.

If this were a Job. A Job has no rolling update. Its pod template is mostly immutable once created, there is no second ReplicaSet to shift onto, and `kubectl rollout` does not work on it. Changing versions would mean deleting the Job and creating a new one, so every pod stops before any new pod starts. A Job also stops once its pods succeed, so it would not have stayed up to serve traffic.

Deployment or Job. Serving work does not finish, so success means staying up, which needs something that holds a replica count steady, replaces pods that die, and changes versions without a gap. Batch work does finish. Success there means each of the eight shards was handled once and then nothing runs, and restarting a validator that already completed would be wrong. That is what `completions: 8` with `restartPolicy: Never` says.